

Portia — Respuesta a la Revisión de Profe Salvador

Punto por punto: qué decía la revisión del 28 de junio, cómo estaba el repositorio antes, y qué se resolvió desde entonces.

Preparado: 03 de julio de 2026

Revisión original de Profe Salvador: 28 de junio de 2026

Resumen ejecutivo

Profe Salvador nos mandó una revisión técnica de 18 páginas el 28 de junio. El diagnóstico central era justo: el proyecto tenía buena documentación de producto, pero le faltaba fortalecer la evidencia técnica — seguridad, pruebas, despliegue y recuperación ante fallos.

Este documento responde punto por punto a esa revisión: qué decía, cómo estaba el repositorio antes, y qué se hizo desde entonces hasta hoy. Todo lo marcado como “resuelto” se verificó en vivo contra GitHub, Supabase y Vercel — no es un resumen de memoria ni una promesa a futuro.

10 resueltos	6 parciales	0 sin atender
------------------------	-----------------------	-------------------------

Recomendaciones prioritarias (§4 de la revisión)

1. Proteger la rama main	
RESUELTO	
ANTES (28-jun) Cualquiera podía pushear directo a main sin Pull Request ni revisión.	AHORA (03-jul) Se activó protección real de la rama: CI obligatorio (build y test de ambas apps), historial lineal, sin force-push ni borrado — y, sobre todo, “enforce admins” activado, que es lo que realmente importa. Se probó con un push directo real: GitHub lo rechazó (“Changes must be made through a pull request”). Antes de esto, incluso con la protección configurada, el dueño del repo podía saltársela — ya no.
2. Separar cambios grandes en Pull Requests distintos	
PARCIAL	
ANTES (28-jun) Un mismo commit podía mezclar panel administrativo, modo offline y notificaciones.	AHORA (03-jul) Cada bump de dependencia y cada corrección quedó en su propio commit, con mensaje explicando qué cambia y por qué. Lo que todavía falta es que esto se vuelva costumbre real vía Pull Request y no solo buena disciplina de commits — con el punto 1 ya resuelto, de ahora en más no queda otra opción: todo cambio nuevo tiene que pasar por PR.

3. Crear documentación técnica (docs/engineering, docs/security, docs/data)

RESUELTO

ANTES (28-jun)

Solo existía documentación de producto (visión, roadmap, features). Nada de arquitectura, seguridad ni despliegue.

AHORA (03-jul)

Las tres carpetas existen con contenido real, no plantillas vacías: docs/engineering/ (arquitectura, despliegue, entorno local, observabilidad, rollback, estrategia de pruebas, troubleshooting), docs/security/ (modelo de amenazas, autenticación y autorización, gestión de secretos, respuesta a incidentes), docs/data/ (modelo de datos, migraciones, respaldo y restauración, retención de datos).

4. Evitar que CLAUDE.md sea la única fuente técnica

RESUELTO

ANTES (28-jun)

Todo el contexto técnico vivía en un solo archivo pensado para el asistente de IA.

AHORA (03-jul)

CLAUDE.md hoy remite a la documentación real en docs/ en vez de duplicarla. Las reglas críticas (no tocar producción directo, no desactivar tests, no introducir secretos) están escritas en archivos que cualquier persona del equipo puede leer, no solo la IA.

5. Sincronizar documentación y código en el mismo Pull Request

RESUELTO

ANTES (28-jun)

Riesgo real de que la documentación quedara desactualizada respecto al código.

AHORA (03-jul)

Se adoptó como práctica: cada migración de base de datos se documenta en el mismo commit que la aplica. El commit de retención de datos de esta semana, por ejemplo, actualiza schema.sql y su explicación al mismo tiempo, no en un commit aparte después.

Seguridad específica de Supabase (§5)

6. RLS obligatorio en todas las tablas

RESUELTO

ANTES (28-jun)

No había confirmación formal de que las 8 tablas tuvieran RLS activo con políticas explícitas.

AHORA (03-jul)

Se verificó directo contra la base de datos real (no el código, la base en vivo): las 8 tablas de public tienen Row Level Security activo. De paso se encontró y cerró un hueco real: la política de la tabla “tareas” dejaba que cualquier conserje autenticado creara o borrara tareas por API directa — ahora eso está restringido solo a administradores.

7. Clave service_role fuera del frontend, del repo y de los logs

RESUELTO

ANTES (28-jun)

La clave se había compartido en texto plano en una conversación previa, para aplicar una corrección puntual.

AHORA (03-jul)

Se probó la clave vieja contra la API real y devolvió 401 Unauthorized — confirma que ya fue rotada. La forma de trabajar ahora usa un conector directo que nunca expone la clave en texto plano, ni en el código ni en el chat.

8. Validar y normalizar el RUT también en el backend, no solo en el frontend

RESUELTO

ANTES (28-jun)

La validación del dígito verificador corría únicamente en el formulario de React. Un request directo a la API podía saltársela sin problema.

AHORA (03-jul)

Se confirmó que la tabla visitas tiene una restricción CHECK a nivel de base de datos que exige el formato correcto de RUT con guion y dígito verificador. La validación existe en el frontend por comodidad del usuario, pero la que realmente importa vive en la base de datos: un request directo con un RUT mal formado es rechazado por Postgres, sin depender de que nadie se acuerde de validar en el camino.

9. Migraciones versionadas, restricciones (FK, NOT NULL, UNIQUE, CHECK) y respaldos probados

PARCIAL

ANTES (28-jun)

No había evidencia de restricciones consistentes ni de que algún respaldo se hubiera restaurado alguna vez.

AHORA (03-jul)

Las migraciones están versionadas en schema.sql con historial completo, y las tablas tienen llaves foráneas, NOT NULL y CHECK. La prueba de restauración sí se intentó en serio esta semana, en dos frentes: contra un Postgres local (no funcionó, porque el esquema depende de roles y tablas propias de la plataforma de Supabase que no existen fuera de ella) y contra un segundo proyecto Supabase gratuito (bloqueado, porque la organización ya tiene el máximo de proyectos gratis en uso). Ninguno de los dos caminos se completó todavía.

Nota: Esta sigue siendo la brecha más concreta de todo el informe, y hay que decirlo con la misma honestidad que exigía Profe Salvador: un respaldo sin restauración probada no es un respaldo confiable. Falta o liberar un proyecto gratis de Supabase, o pasar a un plan pago antes del primer piloto real.

Panel administrativo y modo offline (§6-7)

10. El panel admin no debe depender solo de ocultar opciones en la interfaz

PARCIAL

ANTES (28-jun)

No estaba confirmado si las acciones administrativas también se validaban en el servidor.

AHORA (03-jul)

Las acciones sensibles (como invitar a un conserje) sí validan el rol de administrador del lado del servidor antes de tocar la base de datos. Lo que falta es que esa validación viva en una capa central (el middleware) en vez de repetirse acción por acción — hoy funciona, pero depende de que cada acción nueva se acuerde de hacer la misma verificación.

11. Modo offline: estrategia de sincronización, invalidación y resolución de conflictos

PARCIAL

ANTES (28-jun)

No había estrategia documentada ni probada para la cola de sincronización sin conexión.

AHORA (03-jul)

La lógica de sincronización quedó en una librería propia con tests reales: sincroniza sin duplicar registros, reintenta si la sesión expiró, sube fotos que quedaron encoladas. La resolución de conflictos hoy es simple (gana el servidor, los duplicados se ignoran) y está probada, pero falta escribir en un solo lugar la política de cuánto tiempo vive un dato viejo en la cola antes de descartarse.

Seguridad en Electron y en GitHub Actions (§8-9)

12. Endurecimiento de Electron (nodeIntegration, contextIsolation, sandbox, CSP)

PARCIAL

ANTES (28-jun)

No estaba confirmado si la app de escritorio seguía las prácticas mínimas de seguridad de Electron.

AHORA (03-jul)

Se confirmó en el código del proceso principal: nodeIntegration desactivado, contextIsolation y sandbox activados, política de seguridad de contenido aplicada en producción, y bloqueo de navegación hacia orígenes no permitidos. Falta la firma de código de los instaladores (no hay certificado de desarrollador todavía) y un mecanismo de actualizaciones autenticadas.

13. Permiso contents:write solo en el trabajo de release, no de forma global

RESUELTO

ANTES (28-jun)

No estaba confirmado si el flujo de release tenía permisos de escritura más amplios de lo necesario.

AHORA (03-jul)

Se confirmó en el archivo de release: el flujo completo parte con permiso de solo lectura, y el permiso de escritura existe únicamente dentro del paso que publica la release, después de que los builds de Mac y Windows ya terminaron. Coincide exactamente con lo que pedía la revisión.

Pruebas (§10)

14. Pruebas unitarias, de integración y end-to-end

PARCIAL

ANTES (28-jun)

Existían pruebas unitarias sueltas (como la validación de RUT), pero ninguna que probara un flujo completo de la interfaz.

AHORA (03-jul)

Se pasó de 30 a 33 tests. El más importante es nuevo: monta la página de Visitas completa y simula a un conserje real escribiendo en el formulario, hasta confirmar que el dato correcto llega a Supabase. Cubre el registro exitoso, un RUT inválido, y el consentimiento de datos personales sin marcar. Falta repetir el mismo patrón en Encomiendas, Novedades y Tareas, y sumar pruebas end-to-end reales contra un navegador completo.

Documentos, Definition of Done y flujo de trabajo (§11-17)

15. Documentos recomendados en la raíz (README, CONTRIBUTING, SECURITY, CHANGELOG, LICENSE, CODEOWNERS)

RESUELTO

ANTES (28-jun)

No estaba confirmado cuáles de estos archivos existían.

AHORA (03-jul)

README, CONTRIBUTING, SECURITY y CHANGELOG ya existían, junto con un .env.example por cada app. Esta semana se sumaron los dos que faltaban: LICENSE (propietaria, todos los derechos reservados — Portia es un producto comercial cerrado) y CODEOWNERS.

16. Seguir usando la IA como apoyo, nunca como sustituto de la revisión técnica humana

RESUELTO

ANTES (28-jun)

Recomendación de principio general de la revisión, no un punto técnico puntual.

AHORA (03-jul)

Este mismo documento es la mejor prueba de que se sigue esa idea: cada “resuelto” de este informe se chequeó contra el sistema real — GitHub, Supabase, Vercel — antes de escribirse, en vez de asumir que el trabajo anterior estaba bien hecho.

Conclusión

De los 16 puntos de la revisión de Profe Salvador, 10 quedaron completamente resueltos y verificados, y 6 están en camino con una brecha concreta identificada en cada uno. Ninguno sigue sin atender.

Lo que más urge cerrar antes de un piloto real, en orden:

- La prueba real de restauración de un respaldo — hoy la política de backup sigue siendo teórica, no probada.
- Firma de código para los instaladores de Electron — sin esto, cualquier build puede disparar advertencias de seguridad del sistema operativo al usuario final.
- Que la validación de rol de administrador viva en una sola capa central del panel, no repetida en cada acción.

Ninguno de los tres requiere más que unas horas de trabajo puntual. El resto de las brechas abiertas (tests end-to-end completos, política de invalidación de la cola offline) son mejoras continuas, no bloqueadores para un primer piloto.